

As requested, I am assuming the role of World-Class API Architect and Enterprise Integration Specialist to produce the definitive V1 API Contract Specification.

This specification is derived strictly from the canonical artifacts (Product Blueprint, Scope Cut, Domain Specification, Technical Design, Canonical Data Model, Physical Data Design) and introduces zero new business capabilities. It defines the technology-agnostic contractual boundaries, commands, queries, and behaviors required to execute the V1 system.

Here is the complete V1 API Contract Specification.

SECTION 1 — API Philosophy

To guarantee absolute regulatory fidelity, immutability, and transactional integrity, the API layer adheres strictly to the following principles:

- **Command/Query Responsibility Segregation (CQRS):** State-mutating operations (Commands) are strictly separated from data-retrieval operations (Queries). Commands return minimal acknowledgment (or the resulting Aggregate ID) to prevent read-models from being conflated with transactional responses.
- **Idempotency by Default:** All Commands resulting in external side effects (e.g., MH transmission, invalidation submission) must be idempotent. Repeating a command with the same transaction identifier and parameters must yield the same resulting state without duplicating the external effect or corrupting control numbers.
- **Synchronous Business Validation, Asynchronous Transmission:** Composition, validation, and signing are synchronous operations, returning immediate failure or success. External transmission to the MH is inherently asynchronous (queued) to decouple the user experience from MH API instability.
- **Immutable Historical Truth:** Commands cannot target Signed, Submitted, Accepted, Rejected, or Invalidated documents for modification. Updates are permanently locked post-validation.
- **Audit-Driven Execution:** Every Command executed through this API layer must natively produce a tamper-evident audit entry detailing the exact actor, action, and outcome. If the audit entry fails to persist, the Command must fail.

SECTION 2 — Actor Model

The API enforces strict identity and role-based access control. In V1, exactly two human roles exist. The concept of API-driven Service Accounts is deferred, but the design anticipates programmatic actors.

Actor: Owner

- **Permissions:** Full administrative and operational control. Can execute all configuration, catalog management, and document lifecycle commands.
- **Restrictions:** Cannot permanently delete historical evidence, audit entries, or manipulate transmission records.
- **Visibility Boundaries:** Global visibility over all data within the single-issuer deployment.

Actor: Biller

- **Permissions:** Operational execution only. Can compose, validate, sign, transmit, and invalidate documents.

- **Restrictions:** Cannot manage users, roles, issuer configurations, or establish contingency episodes.
- **Visibility Boundaries:** Can view all operational data (documents, counterparties, items) but cannot access system configuration or user management.

Future System Actors (Deferred to V1.5/V2)

- *Service Account (API Integrator):* Will require granular policy-based access (e.g., restricted to specific DTE types or establishments).

SECTION 3 — Command Catalog

This catalog enumerates the canonical state-mutating operations.

3.1 Issuance Commands

Command: CreateDraftDocument

- **Purpose:** Initializes a new, empty fiscal document.
- **Inputs:** dteType ('01' or '03'), establishmentCode, posCode.
- **Outputs:** documentId.
- **Validations:** DTE Type must be supported. Actor must have Issuance permissions.
- **Authorization:** Owner, Biller.
- **Side Effects:** Creates a new aggregate instance in DRAFT status.
- **Audit:** DOCUMENT_DRAFTED.

Command: UpdateDraftDocument

- **Purpose:** Modifies lines, references, and receptor data before validation.
- **Inputs:** documentId, Header attributes, Line array, Receptor reference.
- **Outputs:** Acknowledgment.
- **Validations:** Document must be in DRAFT status.
- **Authorization:** Owner, Biller.
- **Side Effects:** Overwrites previous draft state; recomputes totals.
- **Audit:** Not required for every keystroke; audited upon transition.

Command: ValidateDocument

- **Purpose:** Evaluates business rules against the MH schema, captures master data snapshots, and assigns legal identity.
- **Inputs:** documentId.
- **Outputs:** generationCode, controlNumber.
- **Validations:** Document must be DRAFT. Totals must perfectly reconcile. CCF (03) requires strict taxpayer receptor identity.
- **Authorization:** Owner, Biller.
- **Side Effects:** Status becomes VALIDATED. Allocates a monotonic Control Number. Captures IssuerSnapshot, ReceptorSnapshot, and ItemSnapshots.
- **Audit:** DOCUMENT_VALIDATED.

Command: SignDocument

- **Purpose:** Cryptographically freezes the payload, rendering the document legally formed.
- **Inputs:** documentId.
- **Outputs:** Acknowledgment.

- **Validations:** Document must be VALIDATED. Payload must align with assigned schemaVersion.
- **Authorization:** Owner, Biller.
- **Side Effects:** Status becomes SIGNED. Generates the canonical JSON. Creates the EvidenceBundle containing the Signed Payload and Human-Readable Representation (PDF). Freezes all snapshots permanently.
- **Audit:** DOCUMENT_SIGNED.

Command: TransmitDocument

- **Purpose:** Queues a signed document for asynchronous delivery to the tax authority.
- **Inputs:** documentId.
- **Outputs:** transmissionRecordId.
- **Validations:** Document must be SIGNED. No active Contingency Episode can exist.
- **Authorization:** Owner, Biller.
- **Side Effects:** Status becomes SUBMITTED. Opens a TransmissionRecord. Queues transmission task.
- **Audit:** TRANSMISSION_ATTEMPTED.

3.2 Invalidation Commands

Command: RequestInvalidation

- **Purpose:** Initiates the plain voiding of a legally accepted document.
- **Inputs:** targetDocumentId, reasonType (Rescind/Other), justification.
- **Outputs:** invalidationId.
- **Validations:** Target must be ACCEPTED. Target cannot already have a pending or accepted invalidation.
- **Authorization:** Owner, Biller.
- **Side Effects:** Creates an InvalidationEvent in DRAFT status.
- **Audit:** INVALIDATION_REQUESTED.

Command: TransmitInvalidation

- **Purpose:** Submits an invalidation request to the authority.
- **Inputs:** invalidationId.
- **Outputs:** transmissionRecordId.
- **Validations:** Event must be fully populated.
- **Authorization:** Owner, Biller.
- **Side Effects:** Status becomes SUBMITTED. Opens transmission lifecycle.
- **Audit:** TRANSMISSION_ATTEMPTED.

3.3 Contingency Commands

Command: OpenEpisode

- **Purpose:** Declares an offline operational period due to network failure.
- **Inputs:** reasonCode, justification.
- **Outputs:** episodeId.
- **Validations:** No other episode can currently be open.
- **Authorization:** Owner only (Policy constraint).

- **Side Effects:** Episode status becomes OPEN. Subsequent signed documents will route to CONTINGENCY_HELD.
- **Audit:** CONTINGENCY_STARTED.

Command: TransmitNotification (Reconcile)

- **Purpose:** Signals the restoration of connectivity and begins batch processing.
- **Inputs:** episodeId.
- **Outputs:** Acknowledgment.
- **Validations:** Episode must be OPEN.
- **Authorization:** Owner only.
- **Side Effects:** Episode transitions to RECONCILING. Transmits the MH notification. Once accepted, automatically queues all held documents for sequential transmission.
- **Audit:** CONTINGENCY_NOTIFIED.

3.4 Master Data Commands

Commands: CreateCounterparty, UpdateCounterparty, CreateItem, UpdateItem

- **Purpose:** Manage reusable catalogs.
- **Inputs:** Canonical entity attributes.
- **Outputs:** Created/Updated ID.
- **Validations:** Strict MH identity validation for counterparties.
- **Authorization:** Owner only.
- **Side Effects:** Mutates master tables. NEVER cascades to historical snapshots.
- **Audit:** CATALOG_UPDATED.

SECTION 4 — Query Catalog

Data retrieval operations return immutable facts or current configuration states.

Query: SearchDocuments

- **Filters:** Date range (Issue Date), Status, DTE Type, Counterparty Name, Document Identity (Generation Code / Control Number).
- **Sorting:** Issue Date, Creation Timestamp.
- **Pagination:** Cursor-based or Offset/Limit.
- **Visibility:** All users can view.

Query: ViewDocument

- **Inputs:** documentId or generationCode.
- **Outputs:** The full aggregate state, including embedded snapshots, lines, totals, and current status.

Query: ViewEvidenceBundle

- **Inputs:** documentId.
- **Outputs:** List of available artifacts (Payload, PDF, MH Responses, Acceptance Stamp) and their cryptographically verifiable hashes.

Query: ViewTransmissionHistory

- **Inputs:** targetIdentity (Document ID or Invalidation ID).
- **Outputs:** Chronological array of transmission attempts, raw requests/responses (if authorized), and interpreted outcomes.

Query: SearchAuditHistory

- **Filters:** Actor, Action Type, Target Identity, Date Range.
- **Outputs:** Chronological array of tamper-evident entries containing the prior and current hashes.

SECTION 5 — Workflow Contracts

The API enforces state machine transitions rigorously.

- **Draft Lifecycle:** Allows fluid, partial updates. Totals do not need to reconcile until the ValidateDocument command is invoked. Errors during composition are validation errors, not business rule failures.
- **Validation Lifecycle:** ValidateDocument is a strict gate. It must execute synchronously. If successful, the document is locked from further updates unless explicitly transitioned back to Draft (which consumes the assigned Control Number).
- **Signing Lifecycle:** SignDocument is an irreversible command. The generated JSON and visual representation are immediately stored in the EvidenceBundle.
- **Submission Lifecycle:** TransmitDocument places the document in SUBMITTED state. The actual MH interaction occurs out-of-band via a queue worker. The UI must poll ViewDocument or ViewTransmissionHistory to detect the terminal ACCEPTED or REJECTED state.
- **Invalidation Lifecycle:** Must follow a saga-like pattern, though V1 implements plain invalidation. The target document remains ACCEPTED until the InvalidationEvent itself reaches ACCEPTED status.
- **Contingency Lifecycle:** When an episode is declared, all subsequent SignDocument commands automatically route the document to CONTINGENCY_HELD rather than SUBMITTED. Reconciliation enforces strict sequential batching: Notification first, then held documents.

SECTION 6 — Error Model

The API returns explicit error categories to guide UI or ERP integration logic.

- **Validation Errors (400 Bad Request):** Structural failures. Missing required fields, invalid tax identity formats, mismatched totals during draft composition.
- **Business Rule Violations (422 Unprocessable Entity):** State machine violations. E.g., attempting to sign a draft, attempting to invalidate a rejected document, or attempting to submit while a contingency episode is open.
- **Authorization Failures (403 Forbidden):** An actor attempts an operation outside their role (e.g., a Biller opening a contingency episode).
- **Regulatory Failures (409 Conflict):** An operation fails because a regulatory requirement is unmet. E.g., attempting to use an unsupported schema version.
- **Transmission Failures (502 Bad Gateway / Async Failure):** The MH API is unreachable or returns a transient error. Handled via the Transmission Record; the document remains SUBMITTED pending retry or contingency escalation.
- **Authority Rejections (Terminal Async Failure):** The MH API explicitly rejects a payload. The document transitions to REJECTED. The error details are surfaced in the observations

array.

SECTION 7 — Concurrency Rules

- **Document Editing Concurrency:** Optimistic concurrency control (e.g., ETags or Version Numbers) must be implemented for UpdateDraftDocument to prevent "last-write-wins" overwrites by multiple billers.
- **Control Number Allocation Concurrency:** The API contract guarantees that ValidateDocument will atomically allocate a unique, monotonic Control Number. Concurrent validation requests for the same POS will be serialized by the underlying database engine; the API will block momentarily rather than fail.
- **Transmission Concurrency:** Distributed locks must prevent multiple queue workers from concurrently transmitting the same document. The API enforces idempotency constraints strictly.

SECTION 8 — Audit Requirements

For every command defined in Section 3, the API layer must orchestrate the creation of a tamper-evident audit entry *before* confirming success to the caller.

- **What must be recorded:** ActionType (e.g., "DOCUMENT_SIGNED"), TargetIdentity (e.g., Document ID), OutcomeStatus (Success/Failure).
- **Who performed it:** ActorIdentity and ActorRole, extracted securely from the authentication context (e.g., JWT).
- **When it occurred:** A strictly monotonic, server-generated Timestamp.
- **Evidence requirements:** The entry must include the PreviousHash and calculate the CurrentHash combining the actor, action, timestamp, and previous hash.

SECTION 9 — Integration Boundaries

While V1 is primarily UI-driven, the API contract is designed to cleanly support future ERP integrations.

- **ERP Integration Points:** Integrators will utilize the CreateDraftDocument and ValidateDocument APIs. The API must support idempotency keys supplied by the ERP to prevent duplicate document creation during network retries.
- **Master Data Synchronization:** Counterparty and Item APIs can be invoked via batch or event-driven mechanisms from an ERP. Because the API enforces the Snapshot Law, synchronizing master data via API will *never* retroactively alter historical invoices.
- **Document Ingestion Boundaries:** Deferred in V1, but the API design explicitly separates outbound issuance (IssuanceRequest) from inbound supplier documents (ReceivedDocument), preventing conflation of fiscal directions.